

THEORY EXAM AI

To ace your AI quiz tomorrow and perfectly align with your professor's unique testing style, we need to focus heavily on the mathematical and algebraic boundaries of Alpha-Beta pruning, as well as the exact node-by-node execution of search algorithms.

Here are three Alpha-Beta pruning questions designed specifically to mimic your professor's favorite exam patterns, followed by the exact hour-by-hour sequence you must study to score full marks.

3 Alpha-Beta Pruning Practice Questions

Question 1: The Algebraic Pruning Limit (The Professor's Favorite) Consider a 3-ply game tree where the Root (Node A) is a **MAX** agent.

- A has two **MIN** children: B (left) and C (right).
 - B has two **MAX** children: D (left) and E (right).
 - C has two **MAX** children: F (left) and G (right).
 - The terminal leaf values are:
 - Children of D: 4 and 3.
 - Children of E: x (left child) and 5 (right child).
 - Children of F: 8 and 6.
 - Children of G: y (left child) and 7 (right child).
- Tasks:**
1. Mathematically determine the range of values for x that will cause the right child of Node E (leaf 5) to be pruned by the Alpha-Beta algorithm.
 2. Assuming $x = 1$, mathematically determine the range of values for y that will cause the right child of Node G (leaf 7) to be pruned. (*Hint: Remember that pruning occurs when a node's value becomes $\leq \alpha$ for MIN, or $\geq \beta$ for MAX.*)

Question 2: Perfect vs. Worst-Case Move Ordering Your professor loves testing the "why" behind the algorithms. Assume an Alpha-Beta search tree with a branching factor of $b = 4$ and a maximum depth of $m = 4$ ply. **Tasks:**

1. Calculate the exact number of terminal nodes evaluated in the absolute worst-case scenario (where Alpha-Beta effectively devolves into standard Minimax).
 2. Calculate the exact number of terminal nodes evaluated if you achieve **perfect move ordering** (where the best moves are always examined first).
 3. Explain mathematically why perfect move ordering changes the effective branching factor from b to $\sqrt[b]{b}$.
-
-

Your Exact Study Sequence (The 30/30 Mid-Sem Roadmap)

Since your quiz is tomorrow, you must study out of syllabus order. Attack Chapter 6 first, then loop back to Chapters 3 and 4.

Step 1: Conquer the Quiz (Chapter 6: Adversarial Search)

- **Focus:** Minimax and Alpha-Beta Pruning.
- **Action:** Draw out the tree for Question 1 above. Write down the α (at least) and β (at most) values explicitly at every single node.
- **Memorize:** The exact Alpha-Beta inequalities. Pruning happens at a MAX node if its value $v \geq \beta$. Pruning happens at a MIN node if its value $v \leq \alpha$.

Step 2: Uninformed Search & Exact Tracing (Chapter 3)

- **Focus:** BFS, DFS, Depth-Limited, and Iterative Deepening.
- **Action:** Do not just read the text. Draw a tree with a branching factor of 2. Write out the exact textbook **frontier** (queue/stack) and **reached** (visited) sets for every step.
- **Crucial Detail to Memorize:** BFS checks the goal state *before* adding a node to the FIFO queue (Early Goal Test), whereas DFS checks the goal state *after* popping it from the LIFO stack (Late Goal Test).
- Add a bullet point to Step 2 that forces you to write out the textbook's 5 formal problem components before tracing any graph: **States, Initial state, Actions, Transition model (Result), and Action cost**
- **Add Uniform-Cost Search (UCS) to your tracing practice. You must know how to trace a graph using path costs ($g(n)$) instead of depth, because your past assignments specifically test optimal step costs (like the Syracuse to Chicago mileage map). Remember that UCS spreads out in waves of uniform path-cost**

Step 3: Informed Search & Heuristic Math (Chapter 3)

- **Focus:** A* Search and Heuristic properties.
- **Action:** You must be able to prove properties mathematically. Write down the definitions of an **Admissible** heuristic (never overestimates the cost: $h(n) \leq h^*(n)$) and a **Consistent** heuristic (obeys the triangle inequality: $h(n) \leq c(n, a) + h(n')$).
- **The Professor's Twist:** Review how to evaluate custom heuristic functions algebraically, such as $f(n) = (2 - w)g(n) + wh(n)$. You must be able to prove what values of w maintain admissibility.
-
- Ensure the *Tree Search Optimality Proof* is on your memorization list. You need to be able to mathematically prove, using a proof by contradiction, that if an algorithm returns a path with cost $C > C^*$, it violates the condition $f(n) \leq C^*$, where C^* is the cost of the optimal path.
-

Step 4: Local Search (Chapter 4)

- **Focus:** Optimization algorithms without path memory.
- **Action:** Write the exact pseudocode for **Simulated Annealing** from memory. You must know the exact probability formula $e^{\Delta E / T}$ used to accept worse moves, and how the "temperature" T helps escape local maxima.
- **Contrast:** Be able to clearly write the difference between steepest-ascent hill climbing (choosing the absolute best neighbor) and first-choice hill climbing (choosing the first randomly generated better neighbor).

Step 5: Genetic Algorithms (Chapter 4)

- **Focus:** Applying GA to routing or real-world problems.
- **Action:** Take a 5-city Traveling Salesperson Problem. Write out:
 1. The chromosome representation (e.g., string of city indices).
 2. The fitness function (e.g., $1 / \text{Total Distance}$).
 3. Execute a manual **Crossover** (splitting two parent strings and swapping).
 4. Execute a manual **Mutation** (randomly swapping a value).

Stick to this sequence tonight. Start immediately with Step 1 and physically write out the α and β variables for the practice question. Good luck on the quiz tomorrow!

QUIZ AI

Since your quiz tomorrow is in **MCQ (Multiple Choice Question)** format, we need to completely pivot our strategy. For an MCQ quiz, your professor won't ask you to write out pseudocode or manually trace a 5-level search tree.

Instead, MCQs test your memory of **algorithm properties, time/space complexities, mathematical formulas, and edge cases**. You need to focus on the exact definitions and comparative tables from the textbook.

Here is your revised, rapid-fire roadmap to study for the MCQ quiz tonight:

Hour 1: Search Algorithm Complexities & Properties (Chapter 3)

MCQs love to ask "Which algorithm is optimal?" or "What is the space complexity of X?". You must memorize the evaluation criteria of search algorithms.

- **Breadth-First Search (BFS):** Space complexity is $O(b^d)$ (exponential, which is its biggest flaw). It is only cost-optimal if all step costs are identical.
- **Uniform-Cost Search (UCS):** Explores based on path cost $g(n)$. Time/Space complexity is $O(b^{1+\lfloor C^* \epsilon \rfloor})$.
- **Depth-First Search (DFS):** Space complexity is only $O(bm)$ (linear, which is its biggest advantage). It is neither complete (in infinite spaces) nor optimal.
- **Iterative Deepening Search (IDS):** The ultimate hybrid. It has the linear memory $O(bd)$ of DFS, but is complete and optimal like BFS.

Hour 2: Heuristics and A* Search (Chapter 3)

Expect true/false or conceptual MCQs on heuristics.

- **Admissibility vs. Consistency:** An **admissible** heuristic *never overestimates* the cost to the goal ($h(n) \leq h^*(n)$). A **consistent** heuristic obeys the triangle inequality: $h(n) \leq c(n, a, n') + h(n')$.
- **MCQ Trick:** Every consistent heuristic is admissible, but not every admissible heuristic is consistent.
- **Domination:** If heuristic $h_2(n) \geq h_1(n)$ for all nodes, then h_2 **dominates** h_1 . A dominating heuristic is better because A* will expand fewer nodes.

- **A Evaluation Function:** $f(n) = g(n) + h(n)$. A* with a consistent heuristic is "optimally efficient" (no other algorithm using the same heuristic expands fewer nodes).
- **Weighted A*:** Uses $f(n) = g(n) + W \times h(n)$ where $W > 1$. This makes the search faster but sacrifices optimality (returns a "satisficing" solution).

Hour 3: Local Search & Optimization (Chapter 4)

This chapter is highly testable in MCQs because of the specific terminology and formulas.

- **Hill Climbing:** Know the reasons it gets stuck: **Local maxima** (a peak lower than the global max), **Plateaus** (flat areas), and **Shoulders** (flat areas with an uphill exit).
- **Simulated Annealing Formula:** Memorize the probability of accepting a *worse* move: $e^{\{-\Delta E/T\}}$.
- **MCQ Trick:** Know how T (temperature) works. If T is high, bad moves are likely accepted. As $T \rightarrow 0$, the probability of accepting a bad move approaches 0, and it becomes greedy hill-climbing.
- **Genetic Algorithms:** Know the three operations: **Selection** (based on fitness function), **Crossover** (recombining parts of two parents), and **Mutation** (randomly flipping bits).

state space landscapes, including the precise definitions of:

- **Local Maximum:** A peak that is higher than its immediate neighbors but lower than the global maximum.
- **Plateau:** A flat, featureless region.
- **Shoulder:** A flat area that has an exit leading uphill.

Hour 4: Adversarial Search / Games (Chapter 6)

Focus on the limits, bounds, and complexities of Minimax and Alpha-Beta pruning.

- **Minimax Complexity:** Time complexity is $O(b^m)$ and space complexity is $O(bm)$.
- **Alpha-Beta Pruning Rules:**
 - α = the value of the best choice found so far for MAX (think: "at least").
 - β = the value of the best choice found so far for MIN (think: "at most").
 - Pruning happens if a value v is worse than the current α or β bounds.
- **Perfect Move Ordering:** If an algorithm explores the absolute best moves first, Alpha-Beta pruning reduces the time complexity from $O(b^m)$ to $O(b^{m/2})$, and the effective branching factor drops from b to $\sqrt{b}$. *This is a very common MCQ question.*

Execution Plan for Tonight:

1. **Ditch the code:** Do not waste time memorizing how to write the exact **EXPAND** function or building full queue tables tonight.

2. **Review the Formulas:** Write down $O(b^d)$, $f(n) = g(n) + h(n)$, $e^{\Delta E/T}$, and $O(b^{m/2})$ on a single piece of paper and memorize them.
3. **Understand the "Why":** For every concept, know *why* it is used. (e.g., Why use IDS? To save memory. Why use Alpha-Beta? To cut the exponent in half. Why use Simulated Annealing? To escape local maxima.)